Multi-Tenant Audit Log + Change Approval System

Status	done

Workload assumptions:
Requirements
User journey
The system
Components at a lower level
- Systems diagram
- Data model
- Storage choice
  - Why I didn't choose a document model
  - Relational model pros and cons
- Api contracts

High level technical flow
Approval workflow states
Multi-tenant isolation
integrity guarantees

Context:

- The platform hosts 1,000+ enterprise customers (tenants).
- Each tenant has multiple environments (dev, staging, prod).
- Thousands of internal and external users perform actions daily.
- Certain actions (e.g. production config changes, permission updates, billing changes) must require approval before taking effect.
- Compliance requires audit data retention for up to 7 years.

- Customers may request export of their audit history at any time.
- Audit logs must be immutable.
- The system must support investigations of audit logs? (query by actor, time range, action, resource).

User Journey:

1. An engineer submits a change request
2. system evaluates policy
3. request is auto-approved or routed for manual approval
4. after approval, the change is executed through a controlled path and then approved
5. every step is recorded in the audit log

ASSUMPTIONS

- Tenants never share change policies at all
- Change records are quite time sensitive (if there is an emergency change or not in sync)
- dev is auto-approved, staging requires local approval, prod requires actual approvals
- Policy rules can be updated directly as long as an audit log is created
- Change requests could require multiple approvals

Workload assumptions:

- events per second (avg and peak) to design for worse case scenarios

  Assuming 4 -5 million daily requests at the log system with a peak of 20% usage → 200K users at any one time (increase in US hours)

  Average Tenant makes 100,00 requests per day

  Scenarios causing peaks (Incidents, bulk operations, major events): add a 20% buffer so **120,000 requests per day**.

- Query patterns
  - Assumption here is that queries are tenant-scoped and also cross-tenants for internal analysis
  - Types of queries
    - Querying for the type of changes made in a certain period
    - Querying by organisation/by user for a certain change
    - Tenants could query when a particular change was made
    - Approval history of changes that were accepted or rejected

Requirements

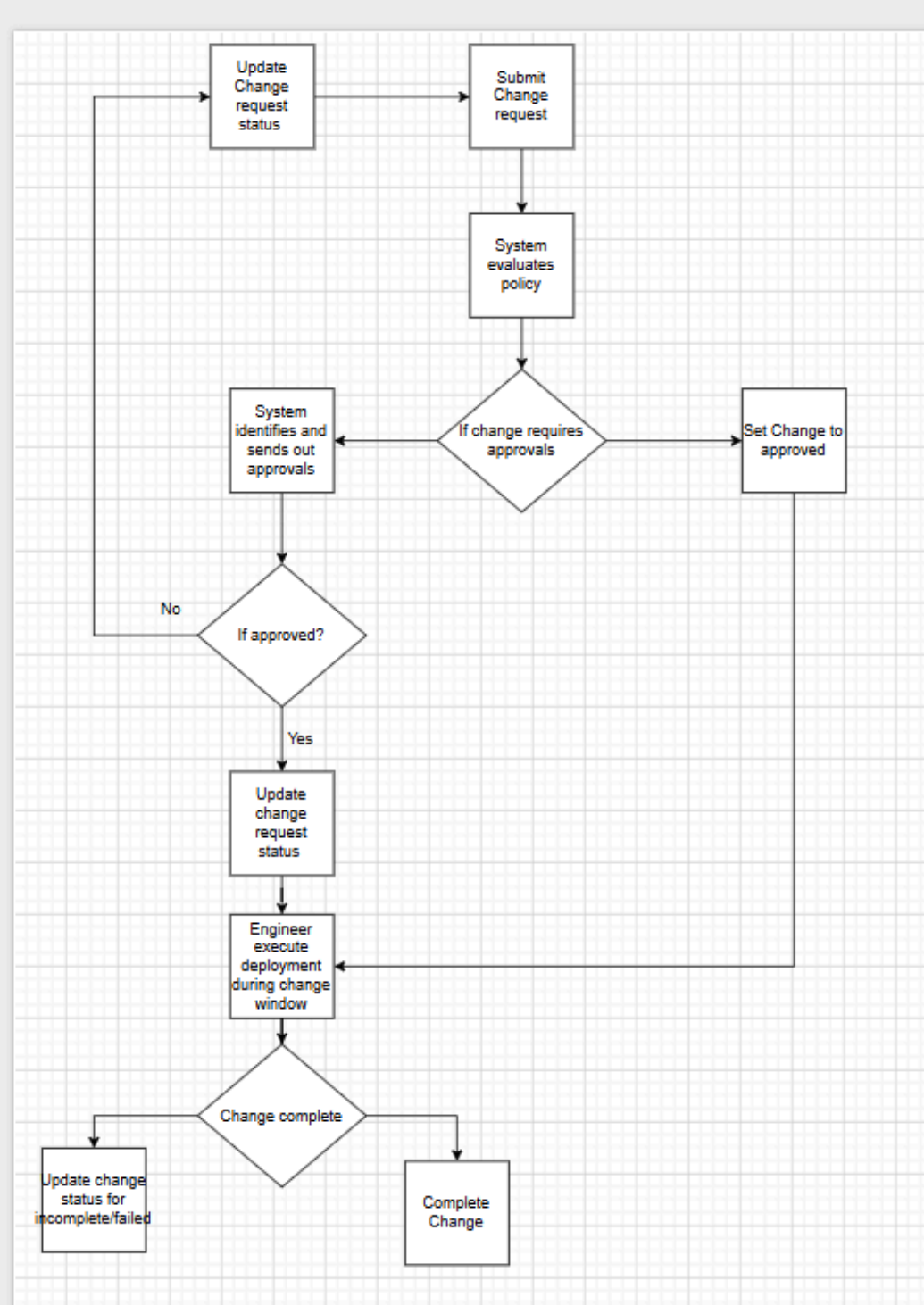
functional requirements

Persona	Requirement	MoSCoW
Tenant engineer	Submit a change to determine whether it needs approval based on the risk, environment, category, and impact.	M
Tenant engineering lead	Create policy approval workflows based on the risk, environment, category, and impact	M
Tenant engineer/lead	Able to carry out investigations of the audit logs	M
Tenant CAB/engineering teams	Ability to approve or reject changes	M
Tenant company	Receive an export of audit history	M
Internal team	Logs shouldn't be editable by tenants or internal staff	M
Internal team	Ability to analyse data across tenants	S
External Tenant	Ability to set up new instances with the system easily	S

Non-functional requirements (choose top 3 and justify)

Scalability/Extensibility	We must be able to gracefully handle a growing number of customers and a growing amount of data on the platform.
Auditability	The data must comply with retention periods and we must be able to trace back the those who approved and made changes.
Performance	• The system is able to support time-sensitive actions such as approvals • Needs to be able to handle heavy querying against the database when exports are requested

User journey



The system

What the system actually needs to do:

1. Create a change record

2. Evaluate the change to see if it needs an approval
3. Manage approval lifecycle if approvals required
4. Update change records based on approved/rejected/auto-approved (audit the approval status)
5. Change Status modifier

(Auditing is also a side effect of these)

Breaking down the responsibilities:

1. Change Writer
2. Change Reader
3. Audit Writer
4. Audit Reader
5. Change Evaluator (2)
6. Approval Lifecycle Handler (3)

Justification: Separate Audit read and writes due to the volume of data and the different systems that will be performing the two tasks aren't overly related.

Components at a lower level

Change Writer

- Inputs: Change number, system/resource ID, change description, tenant id , change category , risk, change state, user ID, environment
- Outputs: Created or updated change record

Change reader:

- Input: Query
- Output: Change record data

Change evaluator

- Inputs: Change Request

- Output: Approval required, approvers

Approval Lifecyle handler

- Inputs: Evaluator Result
- Outputs: State of the Approval

Auditor writer:

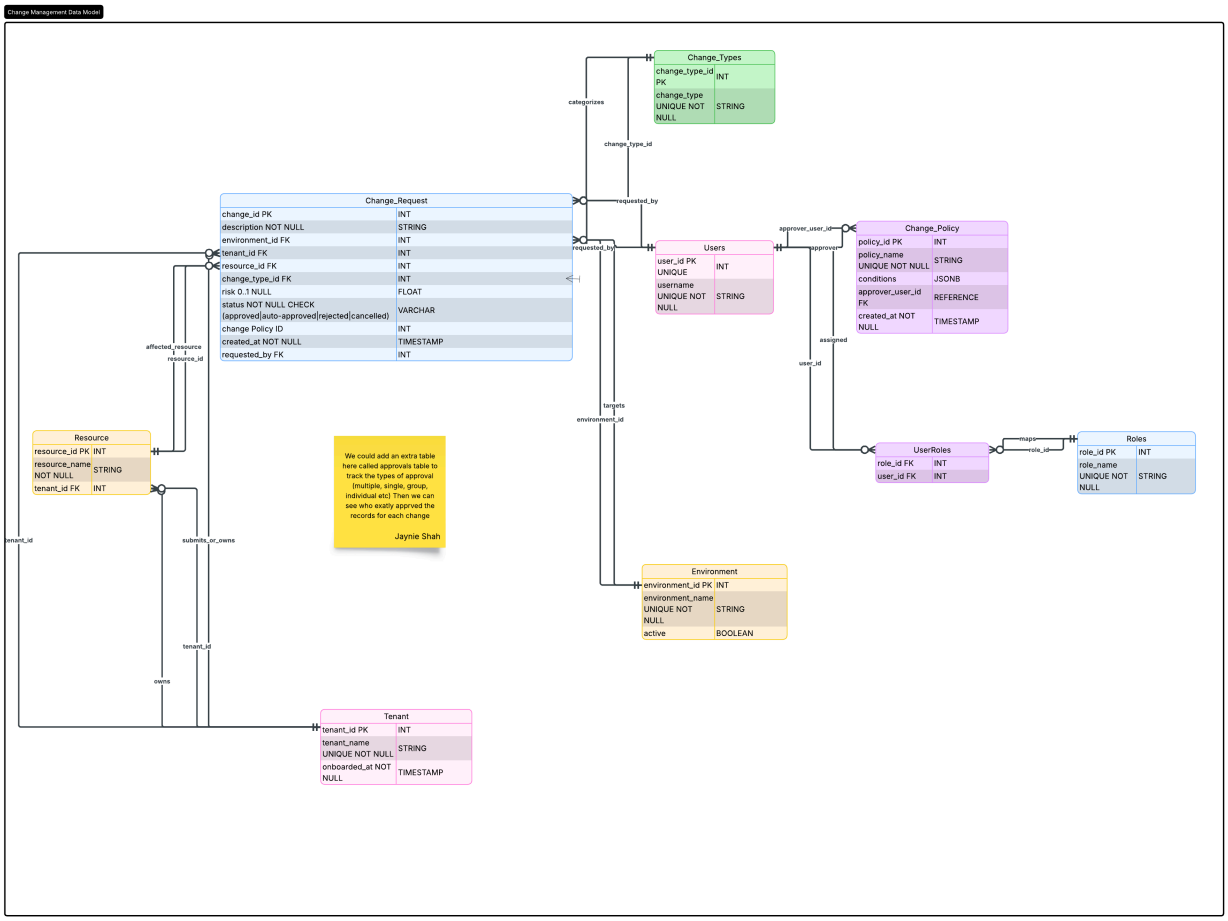
- Inputs: User ID, Tenant ID, Tenant System/Resource ID, Action, version number,
- Outputs: No tangible data approvals

Audit Reader:

- Inputs: Query
- Outputs: Analysed data

Systems diagram

Data model



▼ PSEUDO CODE FOR DATA MODEL

```
Tenant Table
Tenant ID PK INT NOT NULL
Tenant name UNIQUE String NOT NULL
Onboarded at TIMESTAMP NOT NULL

Change Request Table
Change ID PK INT NOT NULL
Description String NOT NULL
Environment FK -> Environment.id NOT NULL
Tenant ID FK -> Tenant.id NOT NULL
Resource ID FK -> Resource.id NOT NULL
Type FK-> Change Types.id Not NULL
Risk ` FLOAT NULL CHECK (`Risk >= 0 AND Risk <= 1`)
```

```
Status VARCHAR(20) NOT NULL CHECK in (`approved`, `auto-ap
proved`, `rejected`,` cancelled')
Created AT TIMESTAMP NOT NULL
Requested By FK -> User.id NOT NULL

Change Policy Table
Policy ID PK INT NOT NULL
Policy Name UNIQUE String NOT NULL
Conditions jsonB
Approvers FOREIGN KEY (user_id) REFERENCES UserRoles(user_
id)
Created AT TIMESTAMP NOT NULL

Environment
Environment ID PK INT NOT NULL
Environment name UNIQUE String NOT NULL
Active BOOLEAN

Change Types
ChangeType ID PK INT NOT NULL
Change Type  UNIQUE String NOT NULL

Users
User ID PK INT NOT NULL UNIQUE
UserName String NOT NULL UNIQUE

Roles
Role ID PK INT NOT NULL
RoleName String NOT NULL UNIQUE

UserRoles
Role ID FK (role_id)
User ID FK (user_id)
```

Storage choice

Why I didn’t choose a document model

Document model relies on everything being chosen

Relational model pros and cons

Api contracts

```
1. POST/PUT Change Request
json:
{
 "created_by" : XXX,
 "created_on": 2026/04/23
 "change_id": CHG23142,
 "change_desciprtion": "Releasing new feature to bulk upload
files"
 "risk" : 5
 "environment" : prod
 "change_type" : code deployment,
 "tenant_id" : Company B
 "resource_id": File Management System
 "status": Pending Approval
 "
}

- **202 Response**

```json
{
  "id": "Change Submitted",
  "status": "submitted",
}
```
```

```
- **Errors**
  - `400 invalid change`
  - `422 unsupported_input`
  - `503 Unsuccessful transaction`

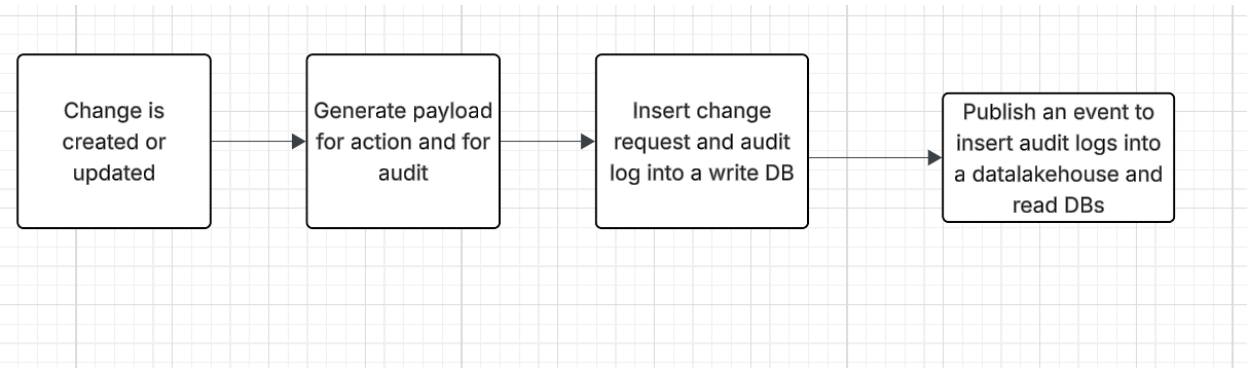
2. POST publish event
json
{tenant_id
version
user_id
action
resource_id
timestamp
}

3.  GET Request Export
json
{tenant_id
query: "CREATED IN 2025 by User ID ABDCF  and action contains
'TEST"
user_id
created on
}
```

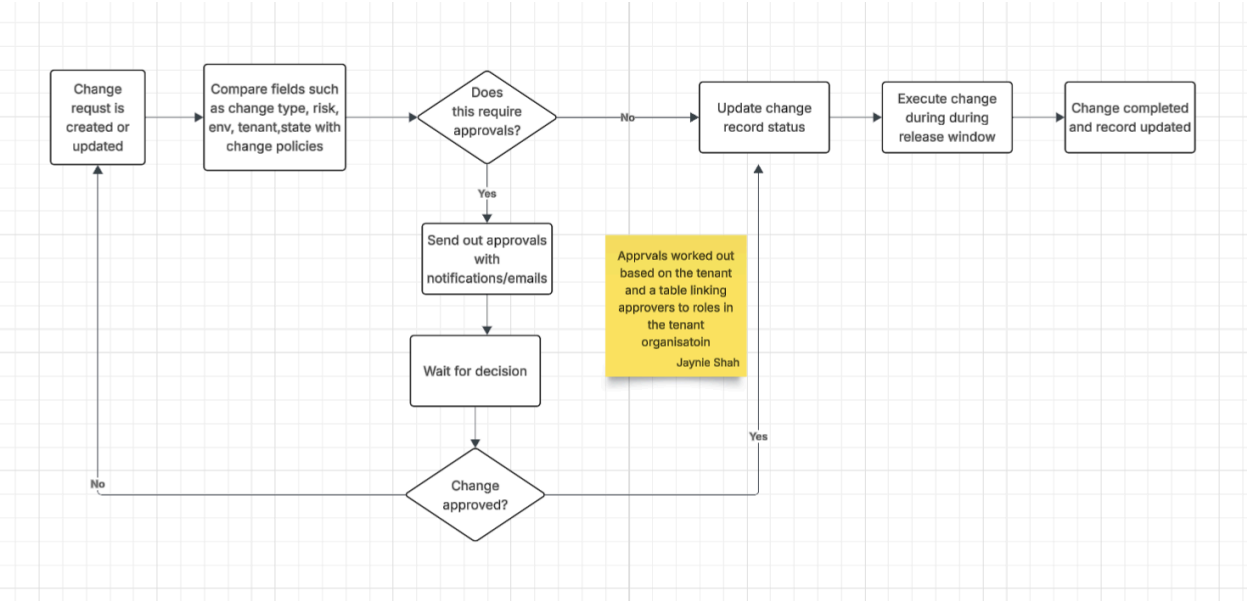
High level technical flow

Key ideas here:

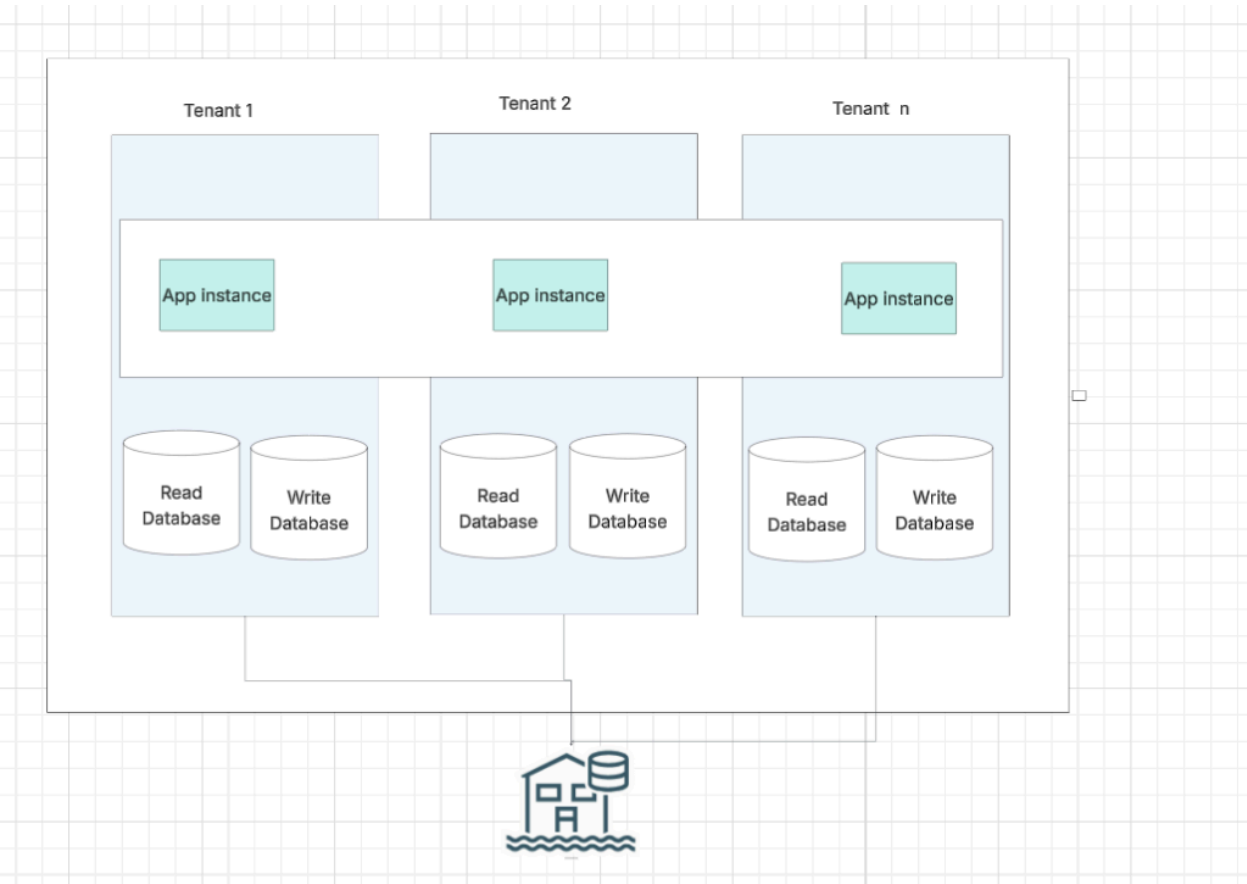
- Event versioning will be a workaround to get around the clocks
- CQRS idea to have a primary write DB and the event
- Data lakehouse aggregates all tenants data into one place so internal analysts can investigate and can also help with the retention of data discussed later



Approval workflow states



Multi-tenant isolation



integrity guarantees

- Audits can never be edited
- Unless a change is ‘draft’ status, it cannot be edited whilst pending approval / has been approved or rejected.

Constraints:

- You cannot delete or modify audit records once written.
- The system must survive regional outage.
- You cannot assume perfect clocks.
- You must justify storage and consistency choices.

Extra

- Audit Log in a separate DB
- As soon as JSONB comes in to place, indexing needs to come into place
- Indexing strategy
- Approvers table → Approval policy , approval strategies to add to data models
- Retention policy
- Change Evaluator, Execute Change (go to into lower level)
- Identify areas of failure
- Sharding policies, request timeout policies, request is rejected